📘 ModelMapper – Complete Notes (Spring Boot / Java)

🔹 What is ModelMapper?

ModelMapper is a Java library used to **automatically map data between objects**, most commonly between:

- **DTO (Data Transfer Object)**
- **Entity (JPA / Hibernate Entity)**

👉 It removes **manual setters/getters code** and makes the application **clean, readable, and maintainable**.

🔹 Why ModelMapper is Needed

Without ModelMapper:

```
EmployeeEntity entity = new EmployeeEntity();
entity.setName(dto.getName());
entity.setEmail(dto.getEmail());
entity.setSalary(dto.getSalary());
```

With ModelMapper:

```
EmployeeEntity entity = modelMapper.map(dto, EmployeeEntity.class);
```

✓ Less code

✓ Fewer bugs

✓ Faster development

🔹 Where ModelMapper is Used

- **Controller → Service**
- **Service → Repository**
- **Entity ↔ DTO conversion**
- **API Request & Response mapping**

🔹 How ModelMapper Works

- Matches **field names**
- Matches **getter/setter methods**
- Automatically converts compatible data types
- Uses **reflection**

🔹 Add Dependency (Maven)

```
<dependency>
  <groupId>org.modelmapper</groupId>
  <artifactId>modelmapper</artifactId>
  <version>3.2.0</version>
</dependency>
```

🔹 Configuration Class

```
@Configuration
public class ModelMapperConfig {
```

```
  @Bean
  public ModelMapper modelMapper() {
    return new ModelMapper();
  }
}
```

✓ Spring creates a **singleton bean**

✓ Can be injected anywhere using constructor injection

🔹 Injecting ModelMapper

```
@Service
public class EmployeeService {

  private final ModelMapper modelMapper;

  public EmployeeService(ModelMapper modelMapper) {
    this.modelMapper = modelMapper;
  }
}
```

🔹 Core map() Method

Two Important Forms

1️⃣ Create New Object

```
EmployeeEntity entity = modelMapper.map(dto, EmployeeEntity.class);
```

- Creates **new instance**
- Used for **CREATE / INSERT**
- Returns mapped object

2️⃣ Update Existing Object

```
modelMapper.map(dto, existingEntity);
```

- Updates the **same object**
- Used for **UPDATE**
- Does **not return** new object

🔹 CREATE Example

```
public EmployeeDTO addEmployee(EmployeeDTO dto)
{
  EmployeeEntity entity = modelMapper.map(dto, EmployeeEntity.class);
  EmployeeEntity saved = employeeRepository.save(entity);
  return modelMapper.map(saved, EmployeeDTO.class);
}
```

✓ New object

✓ INSERT query

🔹 **UPDATE Example (Correct Way)**

```
public EmployeeDTO updateEmployeeById(EmployeeDTO dto, Long id)
{

   EmployeeEntity existing = employeeRepository.findById(id)
         .orElseThrow(() -> new RuntimeException("Employee not found"));

   modelMapper.map(dto, existing);   // update fields

   EmployeeEntity updated = employeeRepository.save(existing);

   return modelMapper.map(updated, EmployeeDTO.class);
}
```

✓ Same entity object

✓ UPDATE query

🔹 ❌ **Common Mistake in UPDATE**

```
modelMapper.map(dto, EmployeeEntity.class);
```

❌ Creates new entity

❌ Does not update fetched entity

❌ May overwrite data

🔹 **One-Line Rule (VERY IMPORTANT)**

Use .class only for CREATE

Use existing object for UPDATE

🔹 **Mapping Entity → DTO**

```
EmployeeDTO dto = modelMapper.map(entity, EmployeeDTO.class);
```

✓ Used in API responses

✓ Hides database structure

🔹 **Mapping List of Objects**

```
List<EmployeeDTO> list = entities.stream()
   .map(e -> modelMapper.map(e, EmployeeDTO.class))
   .toList();
```

🔹 **Handling Null Values**

By default:

- Null values **overwrite** target fields

To skip nulls:

```
modelMapper.getConfiguration()
   .setSkipNullEnabled(true);
```

🔹 **Field Name Mismatch**

DTO: fullName

Entity: name

Custom mapping:

```
modelMapper.typeMap(EmployeeDTO.class, EmployeeEntity.class)
   .addMapping(EmployeeDTO::getFullName, EmployeeEntity::setName);
```

🔹 **Nested Object Mapping**

ModelMapper automatically maps nested objects if names match:

EmployeeDTO → AddressDTO

EmployeeEntity → AddressEntity

🔹 **Advantages**

✓ Less boilerplate code

✓ Clean Service layer

✓ Easy to maintain

✓ Faster development

🔹 **Disadvantages**

❌ Reflection overhead

❌ Hidden mapping errors

❌ Not ideal for very complex mappings

🔹 **Best Practices**

- Always use **DTOs**, not Entities, in Controllers
- Fetch entity first before UPDATE
- Enable skipNull for PATCH APIs
- Keep ModelMapper config centralized

🔹 **Interview One-Liners**

- *ModelMapper is used to convert DTOs and Entities automatically.*
- *For create, map to class; for update, map into existing entity.*
- *It reduces boilerplate code and improves maintainability.*

🔹 **Summary**

- ModelMapper converts objects automatically
- .class → new object
- existing object → update
- Ideal for CRUD-based Spring Boot applications

If you want, I can also provide:

✓ **Handwritten-style notes**

✓ **Interview Q&A**

✓ **Real-world project structure**

Just tell me 👍

📌 ModelMapper – Short Notes (CREATE vs UPDATE)

What map() does

- Converts data between **DTO ↔ Entity**
- Can **create a new object** OR **update an existing object**
- Behavior depends on the **second argument**

🆕 CREATE (Insert)

EmployeeEntity entity = modelMapper.map(dto, EmployeeEntity.class);

- Creates a **NEW Entity object**
- Copies fields from DTO → Entity
- Used when data **does not exist** in DB
- Repository will perform **INSERT**

🔁 UPDATE

modelMapper.map(dto, existingEntity);

- Does **NOT create a new object**
- Updates fields of the **existing entity**
- Used when data **already exists**
- Repository will perform **UPDATE**

❌ Common Mistake

modelMapper.map(dto, EmployeeEntity.class); // during UPDATE

- Creates a **new entity**
- Existing entity remains unchanged
- Can cause **data loss or wrong DB operation**

🧠 One-Line Rule

.class → create new object

existing object → update that object

🗂️ Quick Comparison

Operation	map() Usage	Result
CREATE	map(dto, Entity.class)	New Entity
UPDATE	map(dto, existingEntity)	Updated Entity
Wrong Update	map(dto, Entity.class)	Bug

✅ Best Practice

- **CREATE** → map to Entity.class

- **UPDATE** → map into **fetched entity**
- Always fetch entity first before update

If you want, I can also give **exam-oriented notes** or **interview one-liners** 👍